

İçindekiler

Veri Modeli ve Sahte Veri Planı

- 0. İşaretler
- 0b. ★ SABAH BAKILACAKLAR — bu belgedeki kararlar
 - 1. Genel kurallar — her tabloda geçerli
 - 2. Tablolar ve enum'lar — genel görünüm
 - 3. Tablo tablo
 - 4. Index kararları ✔ (§11 "hangi alanlara index eklendiği, composite index tercihleri")
 - 5. Silme davranışları ✔ (§11 "cascade davranışları")
 - 6. Kişisel veri envanteri (KVKK)
 - 7. Saklama süreleri ve imha
 - 8. Ödev §11'in istediği kararlar — kontrol listesi
 - 9. Sahte veri planı
 - 10. DBML ve migration ilişkisi ✔ (§11 — docs/database.dbml zorunlu)
 - 11. Bu belge nereye bağlanıyor

Veri Modeli ve Sahte Veri Planı

Bu bir TASLAKTIR. Kurulumdan sonra ikiye ayrılır: §1–§8 → `docs/project/data-model.md` · §9 → `docs/project/fake-data-guide.md`. Teslim paketindeki `docs/database-decisions.md` de bu belgeden türetilir (ödev §11 veri tabanı kararlarının dokümente edilmesini **zorunlu** tutuyor).

🚫 **Bu model kopyalanmadı, türetildi.** Ödev §11 son cümlesi: *"Hazır bir şemayı veya bu çalışmayla ilgisiz mevcut bir projeyi uyarlamak yerine, verilen iş ihtiyacına göre veri modeli oluşturmanız beklenmektedir."* Aşağıdaki her tablo, `PRD-taslak.md` içindeki bir iş kuralından doğdu; hangi kuraldan doğduğu her tablonun başında yazılı.

Son güncelleme: 2026-08-26 **Kaynak:** `PRD-taslak.md` · `odev.md` §5, §6, §11, §17, §20, §21 **Şema dosyası (üretilcek):** `apps/api/prisma/schema.prisma`

0. İşaretler

İşaret	Anlamı
✅	Ödevde yazıyor — tartışma yok
★	Ben karar verdim — gerekçesi yanında
⚠️	Varsayım — cevap gelince düzeltilir
?	Senin cevabın şart

0b. ★ SABAH BAKILACAKLAR — bu belgedeki kararlar

★ PRD'deki ★ 1–★ 25 kararları burada **uygulanıyor**; aşağıdakiler ise yalnızca veri modeline ait, **yeni** kararlar.

#	Karar	Neden	Bölüm
★ V1	Birincil anahtar UUID v7 , okunabilir kimlik ayrı kolonda	Sıra numarası adres çubuğunda toplam kayıt sayısını sızdırır ve komşu kaydı tahmin ettirir. v7 zaman sıralı olduğu için v4'ün index dağınıklığı sorununu da çözer	§1.2
★ V2	Enum'lar PostgreSQL yerel enum tipi olarak, değerler metin	Veritabanı geçersiz değeri kendisi reddeder; yeni değer eklemek migration gerektirir — sessizce yeni durum türeyemez	§1.5
★ V3	Yumuşak silme yalnızca <code>users</code> , <code>locations</code> , <code>assets</code> üzerinde	Bunlar başka kayıtların referans verdiği tablolar. İş emri ve geçmiş hiç silinmez ; bildirim ve jeton silinse zarar yok	§1.6
★ V4	<code>is_active</code> ile <code>deleted_at</code> farklı iki şey	<code>is_active</code> bir iş kuralıdır (pasif lokasyonda iş açılmaz), <code>deleted_at</code> bir görünürlük kararıdır (yanlış açılmış kayıt listeden kalkar). Tek alana sıkıştırmak iki gerçeği çakıştırır	§1.6
★ V5	Durum + atama + öncelik + SLA olayları tek <code>work_order_events</code> tablosunda	Hepsi "kim, ne zaman, neyi değiştirdi" kaydı. Ayrı tablolar detay ekranında birleştirme kodu doğurur ve yeni olay türü tablo açmayı gerektirir	§3.6
★ V6	<code>work_orders</code> tablosunda <code>location_id</code> de var (varlıktan türetilabilir olsa da)	Varlık başka binaya taşınırsa geçmiş iş emri o anki binayı göstermeye devam etmeli. Ayrıca lokasyon filtresi join'siz çalışır	§3.5
★ V7	SLA ihlal taraması için kısmi (partial) index	Tarama yalnızca "açık ve henüz ihlal işaretlenmemiş" satırları okur; tam index 400 bin satırlık arşivi de taşırdı. ⚠ Prisma şema dilinde karşılığı yok → migration'a elle SQL yazılacak	§4
★ V8	Yenileme jetonu SHA-256 ile özetlenir, argon2 ile değil	Jeton zaten yüksek entropili rastgele bir değer — sözlük saldırısına açık değil. argon2 her yenilemede boşuna CPU yakar	§3.2
★ V9	<code>holidays</code> tablosunda birincil anahtar tarihin kendisi (doğal anahtar)	Tarih zaten benzersiz ve değişmez; burada vekil anahtar hiçbir şey kazandırmaz. ★ Kural körlemesine uygulanmaz, gerekçesiyle uygulanır	§3.10
★ V10	Günlük özetteki personel yükü <code>jsonb</code> kolonda	Bu veri anlık bir fotoğraf : gösterilir, sorgulanmaz. İlişkiselleştirmek tablo başına satır üretir ve hiçbir sorguyu hızlandırmaz. ⚠ "Son 30 günde X'in yükü" sorusu gelirse tabloya çıkar	§3.9
★ V11	Dosya eki tablosu oluşturulmaz ; tasarımı §3.11'de hazır bekler	Kullanılmayan tablo ödev §32'nin "gereksiz yapı" maddesine takılır. Eklenmesi tek migration + tek modül	§3.11
★ V12	Yorumun yazarı <code>author_id</code> olarak ayrı tutulur, audit <code>created_by</code> 'a emanet edilmez	Audit kolonları altyapıya aittir (merkezî eklenti doldurur); yorumun yazarı iş verisidir . Audit yaklaşımı değişirse iş verisi bozulmamalı	§3.7

#	Karar	Neden	Bölüm
★ V13	E-posta küçük harfe normalize edilerek saklanır	<code>Ali@x.gov.tr</code> ile <code>ali@x.gov.tr</code> iki ayrı hesap olmamalı. Normalizasyon uygulamada yapılır, benzersizlik kısıtı düz kolona konur	§3.1
★ V14	<code>created_by</code> ilk yönetici kaydında boş olabilir	Sistemi kuran ilk kaydın oluşturucusu yok — "tavuk mu yumurta mı". Kolon <code>null</code> kabul eder ve bunun tek meşru sebebi budur	§1.4
★ V15	Sahte veri tarihleri <code>SEED_NOW</code> 'a görelî üretilir	Sabit tarihle tohumlanan demo bir ay sonra bayatlar ("SLA'sı bugün dolacak" listesi boşalır). Görelî üretim demoyu her gün taze tutar; test sabit <code>SEED_NOW</code> vererek determinizmi korur	§9.4
★ V16	Tohumlama, veritabanında sahte olmayan tek bir satır görürse durur	"Yanlışlıkla canlıya seed atma" gerçek ve pahalı bir kazadır. Bayrak zaten var (★ 22), kapıyı da o bayrak koruyor	§9.2

? **Bu belgeye ait açık soru:** ? 1 (kurumun isimlendirme kuralları) — cevap gelmezse §1.1'deki köprü sayesinde **yalnızca şema satırları** değişir.

1. Genel kurallar — her tabloda geçerli

🚫 Bu bölümdeki kararlar bir kez verilir, her tabloda yeniden tartışılmaz.

1.1 İsimlendirme — iki dünya, tek köprü

Katman	Yerleşik pratik	Örnek
TypeScript / NestJS	<code>camelCase</code> alan · tekil tip	<code>dueAt</code> , <code>WorkOrder</code>
PostgreSQL	<code>snake_case</code> kolon · çoğul tablo	<code>due_at</code> , <code>work_orders</code>

🚫 Biri diğerine feda edilmiyor; çeviriyi Prisma'nın `@map` / `@@map` nitelikleri yapıyor:

```
model WorkOrder {                                // kodda: prisma.workOrder
  slaDueAt DateTime @map("sla_due_at") // kodda slaDueAt, veritabanında sla_due_at
  @@map("work_orders")                    // veritabanında tablo adı
}
```

🚫 `@map` ilk migration'dan **ÖNCE** yazılır. Sonradan eklenirse kolon yeniden adlandırma migration'ı gerekir ve canlıda veri taşıma riski doğar.

⚠️ ? 1'in cevabı farklı çıkarsa (örn. tekil tablo adı) **yalnızca sağ taraf** değişir; servis, controller ve ekran kodunun tek satırı dokunulmaz.

1.2 Birincil anahtar ★ V1

- Tip: **UUID** — mümkünse **v7** (zaman sıralı), değilse `cuid()`.
- ★ Neden artan tamsayı değil:
 - `/is-emirleri/145` adresi **toplam kayıt sayısını sızdırır** ve komşu kaydı tahmin ettirir (yetki kontrolü olsa bile bilgi sızıntısıdır).
 - Tohumlama ve testlerde kimlik **önceden bilinebilir** → idempotent `upsert` mümkün olur (§9.3).
 - İleride mobil çevrimdışı kayıt üretilirse çakışma olmaz.
- ★ Neden v7 tercih ediliyor: v4 tamamen rastgeledir ve B-tree index'inde sayfa bölünmesi (page split) yaratır. v7 zamana göre sıralı olduğu için yeni kayıtlar index'in **sonuna** eklenir — artan tamsayının performans avantajı korunur, tahmin edilebilirliği korunmaz.
- 🚫 **İnsana gösterilen kimlik ayırdır:** iş emri numarası (`IE-2026-000148`), lokasyon kodu, varlık kodu. UUID hiçbir ekranda okunması beklenen bir şey değildir.
- ⚠️ ? 1 farklı bir kural getirirse (örn. `bigint identity`) tek migration ile değişir; **iş kodu etkilenmez**, çünkü hiçbir kural kimliğin biçimine dayanmıyor.

1.3 Tarih ve saat ✅ (§11 "tarih ve saat saklama yaklaşımı")

- Tüm zamanlar `timestampz` tipinde ve **UTC** saklanır.
- Ekranda `Europe/Istanbul` 'a çevrilir; çeviri **tek yerde** (arayüz katmanı).
- 🚫 Sistem saati koddan doğrudan okunmaz — `Clock` servisinde alınır (ödev §8), böylece testte sahte saat verilebilir.
- Yalnızca **tarih** anlamı taşıyan alanlar (`installed_at` , `warranty_end_at` , `holiday_date` , `summary_date`) `date` tipindedir. ★ Gerekçe: "garanti 31 Aralık'ta bitiyor" cümlesinin saat dilimi yoktur; `timestampz` yapılırsa saat farkı yüzünden **bir gün kayabilir**.
- ⚠️ Yaz saati uygulaması: Türkiye 2016'dan beri kalıcı UTC+3 kullanıyor, ama hesap yine de saat dilimi kütüphanesi üzerinden yapılır — sabit `+03:00` varsayımı koda gömülmez.

1.4 Audit alanları ✅ (§21)

Her ana tabloda dört alan: `created_at` , `created_by` , `updated_at` , `updated_by` .

Alan	Kim doldurur
<code>created_at</code> / <code>updated_at</code>	Veritabanı varsayılanı ve Prisma <code>@updatedAt</code>
<code>created_by</code> / <code>updated_by</code>	🚫 Elle değil — merkezî Prisma eklentisi, aktif kullanıcıyı istek bağlamından (<code>nestjs-cls</code>) okuyup yazar

★ **Neden merkezi:** Elle doldurulan alan er geç bir serviste unutulur ve o kaydın izi kaybolur — üstelik hata vermez, **sessizce eksik** kalır.

★ V14 `created_by` yalnızca **ilk yönetici** kaydında boştur (onu oluşturan kullanıcı yoktur). 🚫 Başka hiçbir satırda boş olamaz; bu, tohumlama sonrası kontrol sorgusuyla doğrulanır (§9.6).

⚠️ **Audit ≠ geçmiş.** Audit alanları "en son kim dokundu" der; "ne zaman neyi neye çevirdi" sorusunun cevabı `work_order_events` tablosundadır (§3.6). İkisi birbirinin yerine geçmez.

1.5 Enum'ların saklanma biçimi ★ V2 ✅ (§11 açıkça soruyor)

- Prisma `enum` → PostgreSQL **yerel enum tipi**; değerler **metin** görünür.
- ★ Neden sayı değil: veritabanına bakan kişi (DevOps, raporcu, denetçi) `3` 'ün ne olduğunu bilemez. Ayrıca araya yeni değer eklenince sayılar kayar ve **eski kayıtlar yanlış anlam kazanır**.
- ★ Neden düz `text` + `CHECK` değil: yerel enum tipinde veritabanı geçersiz değeri **kendisi** reddeder ve Prisma buradan TypeScript tipi üretir. Bedeli: yeni değer eklemek migration ister — ★ bu bir **özellik**: sistemde sessizce yeni bir durum türeyemez.
- Enum listesi §2'de.

1.6 Yumuşak silme (soft delete) ★ V3 ✅ (§11)

Tablo	Yumuşak silme	Gerekçe
<code>users</code>	✅ <code>deleted_at</code>	Ayrılan personelin geçmiş iş emirleri sahipsiz kalmamalı
<code>locations</code>	✅	Kapanan bina silinirse "bu iş emri hangi binadaydı" cevapsız kalır
<code>assets</code>	✅	Aynı gerekçe
<code>work_orders</code>	🚫 Silinmez	Faaliyet kaydıdır; "arşiv adayı" işareti var, silme yok
<code>work_order_events</code> · <code>work_order_comments</code>	🚫 Silinmez	Değişmez denetim izi
<code>notifications</code> · <code>refresh_tokens</code>	🚫 Gerçek silme serbest	Geçmiş değeri yok
<code>daily_operation_summaries</code> · <code>holidays</code>	🚫	Referans/rapor verisi

★ V4 `is_active` ile `deleted_at` karıştırılmaz:

	<code>is_active</code>	<code>deleted_at</code>
Ne	İş kuralı: pasif lokasyonda yeni iş açılmaz ✅ §5.2	Görünürlük: kayıt listelerde görünmez
Kim kullanır	Operasyon — bina geçici kapandı	Yönetici — kayıt yanlış açılmıştı
Geçmişe etkisi	Yok, mevcut işler devam eder	Yok, geçmiş kayıtlar bağlı kalır

⚠️ **Bedeli:** Her sorguya "*silinmemiş olanlar*" koşulu eklemek gerekir; unutulursa hata vermez, **sessizce yanlış veri** gösterir. Bu yüzden koşul tek tek yazılmaz, merkezî bir filtreyle uygulanır ve `deleted_at` index'lenir.

1.7 Eşzamanlılık ✅ (§20)

- Kullanıcı tarafından güncellenen her ana tabloda `version` (int) kolonu.
- Güncelleme `WHERE id = ? AND version = ?` ile yapılır ve `version` bir artar. Etkilenen satır sayısı 0 ise **başkası araya girmiştir** → `409 Conflict`.
- ★ Neden iyimser (optimistic): Bu sistemde çakışma **nadir**; her güncelleme için satır kilitlemek (pessimistic) nadir bir olayın bedelini **her isteğe** ödetir. Ayrıca kilitle satır bekleyen istekler birikince tüm sistem yavaşlar.
- 🚫 `version` uygulama tarafında elle artırılmaz; güncelleme ifadesinin parçasıdır — iki adım olsaydı arada yine yarış olurdu.

1.8 Null kullanımı ★

`null` **tek** anlama gelir: "*bu bilginin değeri henüz yok veya bu kayıt için uygulanmaz.*"

🚫 Yasak kullanımlar: `null` 'ı "boş metin" yerine kullanmak, `null` 'ı "hayır" yerine kullanmak, `null` 'ı "sıfır" yerine kullanmak.

★ Kural: **Boolean alanlar `null` olmaz** — `NOT NULL DEFAULT false`. Üç değerli mantık (doğru/yanlış/bilinmiyor) sorguları sessizce bozar: `WHERE is_internal = false` koşulu `null` satırları getirmez.

1.9 Metin uzunlukları ★

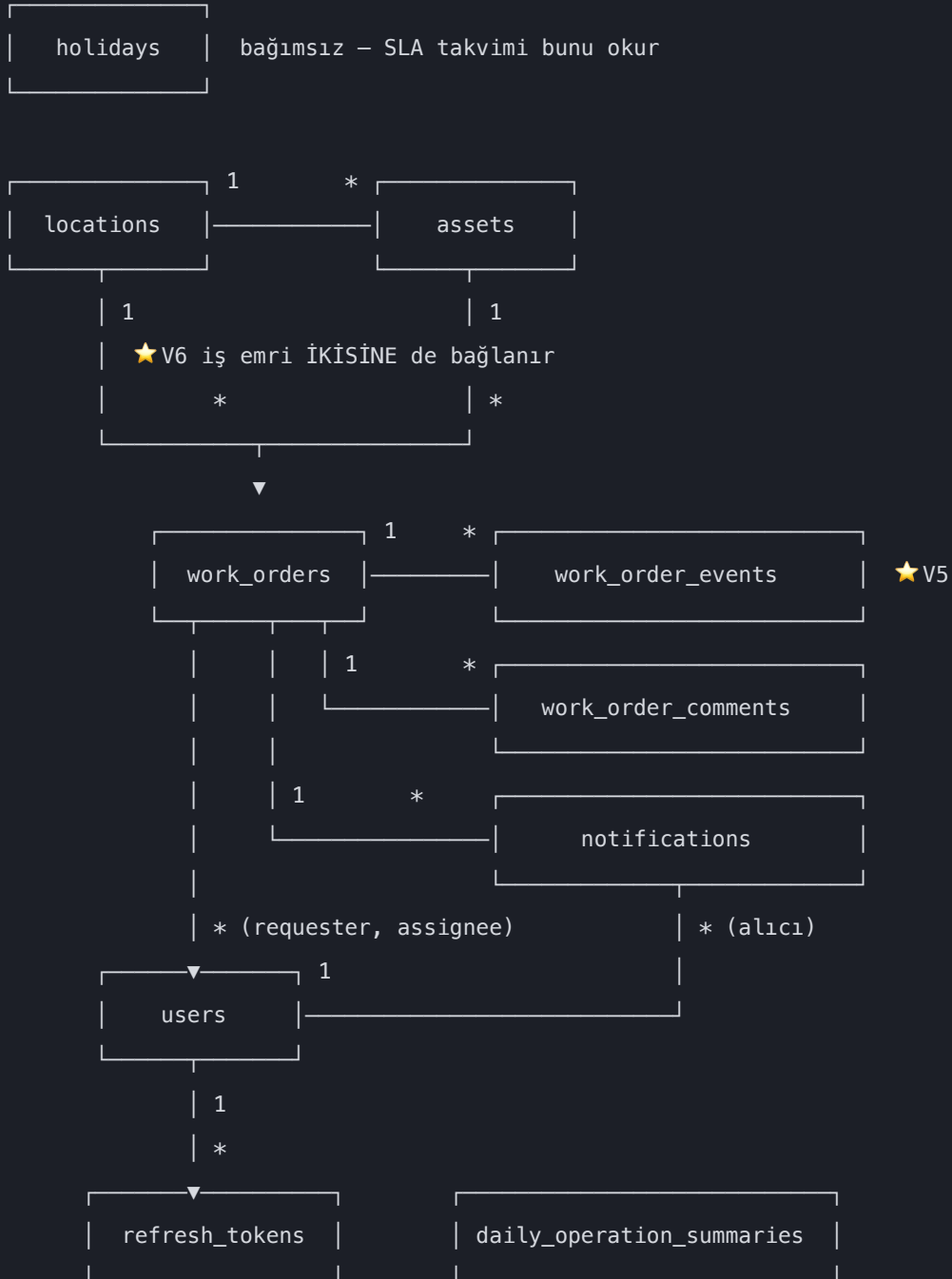
Her metin kolonunda üst sınır var (`text` sınırsız kullanılmaz), sınır Zod şemasındaki sınırla **aynı sayıdır**.

★ **Neden iki yerde de sınır:** Zod sınırı kullanıcıya **anlamlı hata** gösterir; veritabanı sınırı ise API atlanırsa (migration betiği, doğrudan SQL, gelecekteki başka bir istemci) **son savunma hattıdır**.

⚠️ Açıklama alanları `text` tipindedir ama uygulama sınırı vardır (4000) — `varchar(4000)` ile `text` arasında PostgreSQL'de performans farkı yoktur; `text` seçildi ki sınır değişince migration gerekmesin.

2. Tablolar ve enum'lar — genel görünüm

2.1 İlişki şeması



2.2 Tablo listesi

#	Tablo	Ne tutuyor	Hangi PRD maddesinden doğdu
1	users	Personel + kimlik bilgileri	§3, §5.1
2	refresh_tokens	Oturum yenileme jetonları	§5.1
3	locations	Kurumun lokasyonları	§5.2
4	assets	Lokasyona bağlı varlıklar	§5.3
5	work_orders	Talep ve iş emri (tek tablo ★ 1)	§5.4, §5.5, §5.6
6	work_order_events	Durum/atama/öncelik/SLA geçmişi	§5.5, §5.7
7	work_order_comments	Yorumlar ve dahili notlar	§5.7
8	notifications	Sistem içi bildirimler	§5.8
9	daily_operation_summaries	Günlük özet işinin çıktısı	§5.9, §5.10
10	holidays	Resmî tatiller (SLA takvimi)	§5.6
—	(work_order_attachments)	🚫 Açılmıyor — tasarımı §3.11'de	§2 kapsam dışı

2.3 Enum'lar ★ V2

Enum	Değerler	Ekranda
<code>user_role</code>	<code>ADMIN</code> · <code>OPS_MANAGER</code> · <code>TECHNICIAN</code> · <code>REQUESTER</code>	Yönetici · Operasyon Sorumlusu · Teknik Personel · Talep Sahibi
<code>location_type</code>	<code>BUILDING</code> · <code>WAREHOUSE</code> · <code>FACILITY</code> · <code>FIELD</code> · <code>OTHER</code>	Bina · Depo · Tesis · Saha · Diğer
<code>asset_type</code>	<code>GENERATOR</code> · <code>HVAC</code> · <code>ELEVATOR</code> · <code>VEHICLE</code> · <code>PUMP</code> · <code>ELECTRIC_PANEL</code> · <code>IT_EQUIPMENT</code> · <code>OTHER</code>	Jeneratör · İklimlendirme · Asansör · Araç · Pompa · Elektrik panosu · Bilişim donanımı · Diğer
<code>asset_criticality</code>	<code>LOW</code> · <code>NORMAL</code> · <code>HIGH</code> · <code>CRITICAL</code>	Düşük · Normal · Yüksek · Kritik
<code>asset_operational_status</code>	<code>OPERATIONAL</code> · <code>FAULTY</code> · <code>UNDER_MAINTENANCE</code> · <code>OUT_OF_SERVICE</code> · <code>RETIRED</code>	Çalışıyor · Arızalı · Bakımda · Kullanım dışı · Hurdaya ayrıldı
<code>work_order_type</code>	<code>BREAKDOWN</code> · <code>PREVENTIVE</code> · <code>INSPECTION</code> · <code>INSTALLATION</code>	Arıza · Planlı bakım · Periyodik kontrol · Kurulum
<code>work_order_status</code>	<code>REQUESTED</code> · <code>OPEN</code> · <code>ASSIGNED</code> · <code>IN_PROGRESS</code> · <code>WAITING_PART</code> · <code>RESOLVED</code> · <code>CLOSED</code> · <code>CANCELLED</code>	Talep · Açık · Atandı · İşlemdе · Parça bekliyor · Çözüldü · Kapatıldı · İptal edildi
<code>priority</code>	<code>LOW</code> · <code>NORMAL</code> · <code>HIGH</code> · <code>CRITICAL</code>	Düşük · Normal · Yüksek · Kritik
<code>work_order_event_type</code>	<code>CREATED</code> · <code>CONVERTED</code> · <code>STATUS_CHANGED</code> · <code>ASSIGNED</code> · <code>UNASSIGNED</code> · <code>PRIORITY_CHANGED</code> · <code>SLA_RECALCULATED</code> · <code>SLA_PAUSED</code> · <code>SLA_RESUMED</code> · <code>SLA_BREACHED</code>	—
<code>notification_type</code>	<code>WORK_ORDER_ASSIGNED</code> · <code>STATUS_CHANGED</code> · <code>SLA_REMINDER</code> · <code>SLA_ESCALATION</code> · <code>SLA_BREACHED</code> · <code>COMMENT_ADDED</code>	—

★ Neden `priority` ve `asset_criticality` ayrı enum'lar — değerleri aynı görünse de anlamları farklı: biri *işin* aciliyeti, diğeri *varlığın* önemi. Tek enum'a birleştirmek, ileride birine yeni değer eklendiğinde diğeri de kirletir (★ ödev §8 → Interface Segregation'ın veri tarafındaki karşılığı).

3. Tablo tablo

3.1 **users** — personel ve kimlik

Hangi kuraldan doğdu: PRD §3 (dört rol) · §5.1 (giriş) · ★ 3 (tek tablo).

Kolon	Tip	Null	Neden var
id	uuid	🚫	★ V1
email	varchar(255)	🚫	Giriş kimliği. ★ V13 küçük harfe normalize edilir. Benzersiz
password_hash	varchar(255)	🚫	argon2id çıktısı. 🚫 Düz şifre hiçbir yerde durmaz ✅ §5.1
role	user_role	🚫	✅ §4. ⚠️ V4: kullanıcı tek role sahip
is_active	boolean	🚫	✅ §5.1 pasif kullanıcı giriş yapamaz. Varsayılan true
first_name	varchar(80)	🚫	Ekranlarda "atanan personel" olarak görünüyor
last_name	varchar(80)	🚫	Aynı
phone	varchar(20)	✅	İş telefonu — saha personeline ulaşmak somut bir ihtiyaç
unit	varchar(120)	✅	Birim/müdürlük. ★ 15 serbest metin, tablo değil
employee_no	varchar(20)	✅	Kurum sicil numarası. Benzersiz (dolu olduğunda)
is_seed_data	boolean	🚫	★ 22
created_at · created_by · updated_at · updated_by	—	created_by ✅ (★ V14)	§1.4
deleted_at	timestampz	✅	★ V3
version	int	🚫	§1.7

🚫 **Bilerek toplanmayanlar** (KVKK m.4/1-ç veri minimizasyonu, PRD §5.11): ev adresi ★ 3 · T.C. kimlik numarası · doğum tarihi · cinsiyet · fotoğraf · konum. ★ "Toplanmayan veri sızmaz" — en ucuz güvenlik önlemi budur.

- **İlişkiler:** **created_by** / **updated_by** → **users** (kendine referans, **null** kabul eder ★ V14) · **work_orders.requester_id** ve **assignee_id** ← buraya bakar · **refresh_tokens** , **notifications** , **work_order_comments** ← buraya

- **Index:** `email` (benzersiz) · `employee_no` (benzersiz) · `(role, is_active)` — "aktif teknik personel listesi" atama ekranının en sık sorgusu · `deleted_at`
- **Benzersiz:** `email` · `employee_no`

i Neden tek tablo — "kişisel veri ayrı tabloda dursun" argümanına cevap

KVKK **tablo ayrımı istemiyor**; istediği şey (a) gerekenden fazlasını toplamamak, (b) erişimi yetkiyle sınırlamak, (c) saklama süresi tanımlamak. Üçü de bu tasarımda karşılanıyor: alan listesi kırpıldı, yetki matrisi sunucuda uygulanıyor, saklama §7'de yazılı.

⚠ Ayırmanın **gerçekten** gerektiği durumlar var — ve bu projede hiçbiri yok: (1) sistemde kullanıcı **olmayan** personel bulunması, (2) kimliğin dış sağlayıcıdan (LDAP/SSO) gelmesi, (3) bir kişinin birden fazla hesabı olması. ★ Üçünden biri doğarsa ayırma tek migration ile yapılır.

3.2 **refresh_tokens** — oturum yenileme

Hangi kuraldan doğdu: PRD §5.1 (jeton döndürme + yeniden kullanım tespiti).

Kolon	Tip	Null	Neden var
<code>id</code>	uuid	⊖	
<code>user_id</code>	uuid	⊖	Jetonun sahibi
<code>token_hash</code>	varchar(64)	⊖	★ V8 SHA-256 özeti. ⊖ Jetonun kendisi saklanmaz — veritabanı sızarsa oturumlar ele geçmesin. Benzersiz
<code>expires_at</code>	timestampz	⊖	⚠ 7 gün
<code>revoked_at</code>	timestampz	✓	Dolu ise jeton kullanılmış/iptal edilmiş
<code>replaced_by_id</code>	uuid	✓	★ Hangi yeni jetonla değiştirildi — yeniden kullanım tespitinde zinciri görmeyi sağlar
<code>created_at</code>	timestampz	⊖	

⊖ **Saklanmayanlar** ★ 21: IP adresi, tarayıcı bilgisi, konum.

- **İlişkiler:** `user_id` → `users`, **ON DELETE CASCADE** ★ (jeton kullanıcı olmadan anlamsızdır) · `replaced_by_id` → kendine
- **Index:** `token_hash` (benzersiz — girişteki tek sorgu bu) · `(user_id, revoked_at)` — "bu kullanıcının açık oturumlarını kapat" sorgusu
- ⚠ **Temizlik:** Süresi geçmiş jetonlar bir bakım işiyle silinir; aksi hâlde tablo sonsuza kadar büyür. (Bu, ödevdeki dört işe ek beşinci bir iş değildir — arşiv adayı işiyle aynı çalışmada yapılır.)

3.3 **locations** — lokasyonlar

Hangi kuraldan doğdu: PRD §5.2 (beş işlem + pasif lokasyon kuralı).

Kolon	Tip	Null	Neden var
<code>id</code>	uuid	🚫	
<code>code</code>	varchar(30)	🚫	İnsanın kullandığı kısa kod (<code>KONAK-HZM-01</code>). Benzersiz — silinmişler dahil ★ (aksi hâlde geçmişte iki farklı bina aynı kodu taşır)
<code>name</code>	varchar(200)	🚫	✅
<code>type</code>	<code>location_type</code>	🚫	Filtreleme için
<code>address</code>	varchar(500)	✅	★ Kurumun adresi — kişisel veri değil
<code>description</code>	text	✅	Uygulama sınırı 1000
<code>is_active</code>	boolean	🚫	✅ §5.2 — iş kuralı : pasifse yeni varlık/iş emri yok
<code>is_seed_data</code>	boolean	🚫	★ 22
audit 4 alan + <code>deleted_at</code> + <code>version</code>	—	—	§1.4, §1.6, §1.7

- **İlişkiler**: `assets.location_id` ← buraya (**RESTRICT**) · `work_orders.location_id` ← buraya (**RESTRICT**)
- **Index**: `code` (benzersiz) · (`is_active`, `deleted_at`) — liste ekranının varsayılan filtresi · `name` **GIN** + `pg_trgm` (arama)
- **İş kuralı hatırlatması**: Pasifleştirme **geleceği** kapatır; mevcut iş emirleri akışına devam eder (PRD §5.2 kural 3).

3.4 **assets** — varlıklar

Hangi kuraldan doğdu: PRD §5.3 (altı işlem + altı zorunlu bilgi + iş emri kabul kuralı).

Kolon	Tip	Null	Neden var
<code>id</code>	uuid	🚫	
<code>code</code>	varchar(30)	🚫	Kurum içi tanımlayıcı (<code>JEN-0042</code>). Benzersiz ✅ §5.3
<code>name</code>	varchar(200)	🚫	
<code>location_id</code>	uuid	🚫	✅ §5.3 "bağlı olduğu lokasyon"
<code>type</code>	<code>asset_type</code>	🚫	✅ §5.3 "varlık türü"
<code>criticality</code>	<code>asset_criticality</code>	🚫	✅ §5.3 · ⭐ SLA çarpanı buradan geliyor — bu kolon olmadan §7 karşılanamaz
<code>operational_status</code>	<code>asset_operational_status</code>	🚫	✅ §5.3 · ⭐ 9 iş emri kabul kuralı buna bakıyor
<code>brand</code> · <code>model</code> · <code>serial_no</code>	varchar(80)	✅	✅ §5.3 "tanımlayıcı bilgileri"
<code>installed_at</code> · <code>warranty_end_at</code>	date	✅	✅ §5.3 "bakım bilgileri". §1.3: <code>date</code> , <code>timestampz</code> değil
<code>last_maintenance_at</code> · <code>next_maintenance_at</code>	date	✅	Aynı
<code>maintenance_period_days</code>	int	✅	Periyodik bakım aralığı
<code>is_seed_data</code>	boolean	🚫	
audit 4 alan + <code>deleted_at</code> + <code>version</code>	—	—	

- **İlişkiler:** `location_id` → `locations` (**RESTRICT**: varlığı olan lokasyon silinemez) ·
`work_orders.asset_id` ← buraya (**RESTRICT**)
- **Index:** `code` (benzersiz) · `(location_id, operational_status)` — varlık listesinin en sık filtre çifti ·
`criticality` · `deleted_at` · `(name, code)` **GIN** + **pg_trgm** (arama)
- ⭐ **Kritiklik değişirse açık iş emirlerinin SLA'sı yeniden hesaplanmaz** (PRD §5.3 kural 6) — SLA, iş emri açıldığı andaki taahhüttür.

3.5 `work_orders` — talep ve iş emri ⭐ 1

⭐ **Sistemin kalbi.** Ödev §5.4'ün 13 zorunlu alanının tamamı burada.

Hangi kuraldan doğdu: PRD §5.4 (alanlar + numara) · §5.5 (durum makinesi) · §5.6 (SLA) · §5.9 (arşiv).

A) Kimlik ve sınıflandırma

Kolon	Tip	Null	Neden var
id	uuid	⊖	
number	varchar(20)	⊖	✅ §5.4 "benzersiz ve okunabilir numara" — IE-2026-000148 . Benzersiz ★ 13
type	work_order_type	⊖	✅ · ★ 6 SLA politikasını bu seçiyor
status	work_order_status	⊖	✅ §6
priority	priority	⊖	✅ · SLA temel süresi bundan
title	varchar(200)	⊖	✅
description	text	⊖	✅ (uygulama sınırı 4000)

B) Bağlantılar

Kolon	Tip	Null	Neden var
location_id	uuid	⊖	✅ · ★ V6 varlıktan türetilabilir olmasına rağmen ayrıca saklanıyor
asset_id	uuid	⊖	✅
requester_id	uuid	⊖	✅ "talebi oluşturan kullanıcı"
assignee_id	uuid	✅	✅ "atanan teknik personel" — atanmamışken boş

★ V6 **location_id** neden ayrı duruyor — iki sebep:

- Zaman doğruluğu:** Varlık başka binaya taşınırsa geçmiş iş emri **o işin yapıldığı** binayı göstermeye devam etmeli. Varlıktan okunsaydı geçmiş sessizce değişirdi.
- Sorgu maliyeti:** Lokasyon filtresi ve **(location_id, status)** birleşik index'i join gerektirmez.

⚠ **Bedeli:** İki alan tutarlı tutulmalı. Kural: oluşturma anında **asset.location_id** ile aynı olmalı (doğrulama), sonra **dondurulur**.

C) Zaman ve SLA (PRD §5.6)

Kolon	Tip	Null	Neden var
<code>planned_at</code>	timestampz	✓	★ 7 Planlı bakım/kontrol/kurulumda zorunlu , arızada yasak
<code>sla_policy_code</code>	varchar(40)	✗	★ Hangi politika hesapladı. "Bu işin süresi neden 90 dakika" sorusunun cevabı; canlı incelemede en çok sorulacak yer
<code>sla_start_at</code>	timestampz	✗	★ 2 Talep açılış anı
<code>sla_due_at</code>	timestampz	✗	✓ §5.4 "SLA bitiş zamanı"
<code>sla_remind_at</code>	timestampz	✓	%50 anı
<code>sla_escalate_at</code>	timestampz	✓	%80 anı
<code>sla_paused_at</code>	timestampz	✓	★ 8 duraklatma başlangıcı
<code>sla_paused_total_minutes</code>	int	✗	★ 8 birikmiş duraklama. Varsayılan 0
<code>is_sla_breached</code>	boolean	✗	✓ §15 "SLA ihlali olarak işaretlemelidir"
<code>sla_breached_at</code>	timestampz	✓	İhlal anı

✗ `sla_reminded_at` / `sla_escalated_at` kolonları YOK. "Bu hatırlatma daha önce gönderildi mi" sorusunun cevabı `notifications.dedupe_key` benzersizliğindedir (★ 12). İki yerde tutulsaydı ikisi ayrışabilirdi ve hangisi doğru bilinmezdi.

D) Sonuç ve yaşam döngüsü

Kolon	Tip	Null	Neden var
<code>converted_at</code>	timestampz	✓	TALEP → AÇIK anı ★ 1
<code>resolution</code>	text	✓	✓ §6 "çözüldü durumuna alınırken çözüm açıklaması girilmelidir"
<code>resolved_at</code> · <code>closed_at</code> · <code>cancelled_at</code>	timestampz	✓	Ekranı ve raporda kullanılıyor; durumdan türetilemez çünkü durum ileri geri gidebilir
<code>cancel_reason</code>	varchar(500)	✓	★ İptal gerekçesi zorunlu
<code>is_archive_candidate</code> · <code>archive_candidate_at</code>	boolean/timestampz	✗ / ✓	✓ §15 dördüncü iş ★ 24
<code>is_seed_data</code>	boolean	✗	★ 22
audit 4 alan + <code>version</code>	—	—	✗ <code>deleted_at</code> yok — iş emri silinmez

⚠ **resolved_at** neden ayrı kolon: İş emri **ÇÖZÜLDÜ** den **İŞLEMDE** ye geri dönebilir (sorun tekrarladı). Durumdan okumaya çalışsaydık "ne zaman çözülmüştü" cevabı kaybolurdu. **Son** çözülme anı burada tutulur, **tüm** çözülme denemeleri olay tablosunda.

- **İlişkiler:** dört FK de **RESTRICT** ★ (referans verilen kayıt gerçek silmeyle yok edilemez — zaten hepsi yumuşak siliniyor)
- **Index:** §4'te toplu

3.6 work_order_events — geçmiş ★ V5

Hangi kuraldan doğdu: PRD §5.5 kural 1–3 · §5.7 · ✓ §6 "her durum değişikliği geçmiş kaydı oluşturmaldır", "atama ve görevden alma işlemleri geçmiş kayıtlarıyla takip edilmelidir".

Kolon	Tip	Null	Neden var
id	uuid	⊖	
work_order_id	uuid	⊖	Hangi iş emrine ait
type	work_order_event_type	⊖	Olayın türü
from_status · to_status	work_order_status	✓	Durum değişikliğinde dolu
from_assignee_id · to_assignee_id	uuid	✓	Atama/görevden almada dolu
from_priority · to_priority	priority	✓	Öncelik değişikliğinde dolu
note	varchar(1000)	✓	İptal gerekçesi, yeniden açma sebebi, çözüm özeti
payload	jsonb	✓	★ Olaya özel ek bilgi (yeni SLA tarihi, duraklama dakikası). ⚠ Yalnızca gösterilir, üzerinde sorgu yapılmaz — sorgulanması gereken bir alan çıkarsa kolona terfi eder
created_at · created_by	—	⊖	Kim, ne zaman

⊖ **updated_at** , **updated_by** , **deleted_at** **YOK**. Olay kaydı **değişmezdir** (immutable): yazılır, bir daha dokunulmaz. Değiştirilebilen iz, iz değildir.

★ V5 kararının uzun hâli: neden iki değil tek tablo

Ödev §11 "*işlem geçmişi tabloları*" diyor (çoğul) ve akla ilk gelen tasarım

`work_order_status_history` + `work_order_assignment_history` ikilisidir. Seçmedim, üç sebeple:

1. **Detay ekranı zaten birleştiriyor.** PRD §5.7'deki zaman çizelgesi tek akıştır. İki tablo olsaydı iki sorgu çekilip bellekte tarihe göre birleştirilirdi — hem fazladan kod hem sayfalaması imkânsız bir liste.
2. **Yeni olay türü eklemek tablo açmayı gerektirirdi.** Öncelik değişikliği ve SLA yeniden hesaplaması da geçmişe düşüyor; iki tabloyla dört tablo olurdu ve her biri aynı dört audit alanını tekrarlardı (✓ §9 DRY).
3. **Yazma yolu tek.** Her geçiş `prisma.$transaction` içinde **bir** satır yazıyor; iki tabloya yazan iki ayrı yol olsaydı biri unutulduğunda geçmiş sessizce eksik kalırdı.

⚠ **Bedeli dürüstçe:** Altı kolon (`from_*` / `to_*`) her satırda dolu değil — durum olayında atama kolonları boş. Bu, "geniş ve seyrek tablo" eleştirisine açık bir tasarımdır. ★ Kabul edildi çünkü alternatifin bedeli (dört tablo + birleştirme kodu) daha yüksek ve PostgreSQL `null` kolonları fiilen yer kaplamaz.

★ **Ödevin "çoğul" ifadesi karşılanıyor mu?** Evet: geçmiş **tabloları** `work_order_events` ve `work_order_comments` 'tir; ikisi de ana kayıttan ayrı, ikisi de değişmez. Ödevin istediği şey ayrı **tablo sayısı** değil, geçmişin ana kayıttan ayrı ve izlenebilir tutulmasıdır.

- **İlişkiler:** `work_order_id` → `work_orders` **ON DELETE CASCADE** ★ (olay iş emri olmadan anlamsızdır; iş emri zaten hiç silinmiyor, bu bir **niyet beyanı**)
- **Index:** (`work_order_id`, `created_at`) — detay ekranının tek sorgusu · `type` (rapor)

3.7 `work_order_comments` — yorumlar ★ 11

Hangi kuraldan doğdu: PRD §5.7 · ✓ §16 "*iş emrine yorum eklendiğinde*" bildirim.

Kolon	Tip	Null	Neden var
<code>id</code>	uuid	⊖	
<code>work_order_id</code>	uuid	⊖	
<code>author_id</code>	uuid	⊖	★ V12 Yazarı iş verisidir , audit kolonuna emanet edilmez
<code>body</code>	varchar(2000)	⊖	⚠ 1–2000 karakter
<code>is_internal</code>	boolean	⊖	★ 11 Dahili not — talep sahibi göremez. Varsayılan <code>false</code>
<code>created_at</code> · <code>created_by</code>	—	⊖	

🚫 **updated_at** ve **deleted_at** YOK — yorum düzenlenemez ve silinemez (PRD §5.7 gerekçesi). ★ Bu bir **veri modeli kararıdır**: kolonu koymamak, "düzenlemeyi ekranda kapatmak"tan güçlüdür — API atlansa bile düzenlenemez.

- **İlişkiler**: **work_order_id** → CASCADE · **author_id** → **users** RESTRICT
- **Index**: (**work_order_id**, **created_at**) · (**work_order_id**, **is_internal**) — ★ talep sahibine dönen listede dahili notlar **veritabanı seviyesinde** filtrelenir, bellekte ayıklanmaz

3.8 **notifications** — bildirimler ★ 12

Hangi kuraldan doğdu: PRD §5.8 · ✅ §16 (beş olay + okundu yönetimi + mükerrer engeli) · ✅ §11 "bildirim tekrarlarını engelleyen yapılar".

Kolon	Tip	Null	Neden var
id	uuid	🚫	
user_id	uuid	🚫	Alıcı
type	notification_type	🚫	
work_order_id	uuid	✅	İlgili iş emri (bildirimden detaya gitmek için)
title	varchar(200)	🚫	★ Metin üretildiği anda yazılır, okuma anında hesaplanmaz — iş emri sonradan değişse bile bildirim o günkü gerçeği anlatır
body	varchar(1000)	🚫	Aynı
dedupe_key	varchar(120)	🚫	★ 12 Benzersiz — mükerrer bildirimin tek ve gerçek engeli
is_read · read_at	boolean/timestampz	🚫 / ✅	✅ §16 okundu yönetimi
created_at	timestampz	🚫	

dedupe_key nasıl kuruluyor ★ :

Olay	Anahtar	Neden bu biçim
SLA hatırlatma	<code>wo:{id}:sla-remind</code>	İş emri başına bir kez
SLA escalation	<code>wo:{id}:sla-escalate</code>	Bir kez
SLA ihlali	<code>wo:{id}:sla-breach</code>	Bir kez — tarama işi 5 dakikada bir çalışsa da
Atama	<code>wo:{id}:assigned:{event_id}</code>	Her atama olayı için bir kez (yeniden atanırsa yeni bildirim doğru)
Durum değişikliği	<code>wo:{id}:status:{event_id}</code>	Aynı mantık
Yorum	<code>wo:{id}:comment:{comment_id}:{user_id}</code>	Yorum başına, alıcı başına bir kez

★ **Alıcı kimliği anahtarın içinde** olduğu için tek bir global benzersiz index yeterli oluyor — kullanıcı başına ayrı kısıt gerekmiyor.

🚫 **Neden uygulama kontrolü yetmez:** "Bu bildirim var mı? Yoksa ekle" iki iş aynı anda çalışırsa **ikisi de** "yok" görür ve ikisi de ekler. Buna yarış koşulu (race condition) denir; tek kullanıcıli testte asla görünmez, yük altında ortaya çıkar. Veritabanı benzersizlik kısıtı bu yarış yapısal olarak kaybettirir: ikinci **INSERT** hata alır, iş onu **sessizce yutar** (idempotency).

- **İlişkiler:** `user_id` → CASCADE · `work_order_id` → CASCADE
- **Index:** `dedupe_key` (benzersiz) · `(user_id, is_read, created_at DESC)` — zil ikonundaki "okunmamış" sayacının ve bildirim listesinin tek sorgusu

3.9 **daily_operation_summaries** — günlük özet ★ 17

Hangi kuraldan doğdu: PRD §5.9 iş 3 · §5.10 · ✅ §15 (beş gösterge).

Kolon	Tip	Null	Neden var
<code>id</code>	uuid	🚫	
<code>summary_date</code>	date	🚫	Benzersiz ★ — işin idempotentliğinin veritabanı seviyesindeki kanıtı
<code>open_count</code>	int	🚫	✅ Açık iş emri sayısı
<code>breached_count</code>	int	🚫	✅ SLA ihlali olan
<code>critical_open_count</code>	int	🚫	✅ Kritik iş emri
<code>completed_count</code>	int	🚫	✅ Gün içinde tamamlanan
<code>workload</code>	jsonb	🚫	✅ Teknik personel bazında aktif iş yükü. ★ V10
<code>generated_at</code>	timestampz	🚫	İşin çalıştığı an — gecikmeyi görmek için
<code>created_at</code>	timestampz	🚫	

★ `summary_date` benzersiz olduğu için iş iki kez çalışsa da ikinci satır oluşmaz — `upsert` kullanılır ve sayılar tazelenir. Ödev §15'in "*joblar idempotent olmalıdır*" maddesi burada **iddia değil, kısıt** olarak duruyor.

⚠️ `workload` neden `jsonb` ★ V10: bu veri anlık bir fotoğraf; gösterilir, üzerinde sorgu yapılmaz. İlişkiselleştirmek her gün için personel sayısı kadar satır üretir ve hiçbir sorguyu hızlandırmaz. "Son 30 günde X'in yükü" sorusu gelirse ayrı tabloya çıkar — o zaman gerçekten sorgulanan bir veri olur.

3.10 `holidays` — resmî tatiller ★ 16

Hangi kuraldan doğdu: PRD §5.6 karma takvim (mesai hesabı).

Kolon	Tip	Null	Neden var
<code>holiday_date</code>	date	🚫	★ V9 Birincil anahtar — doğal anahtar
<code>name</code>	varchar(120)	🚫	"Cumhuriyet Bayramı"
<code>is_half_day</code>	boolean	🚫	⚠️ V3 Arife günleri
<code>half_day_end_time</code>	time	✅	Yarım günde mesainin bittiği saat (⚠️ 13:00)
<code>is_seed_data</code>	boolean	🚫	
<code>created_at</code>	timestampz	🚫	

★ V9 **Neden UUID değil de tarihin kendisi:** Tarih zaten benzersiz, değişmez ve anlamlı. Vekil anahtar (surrogate key) burada fazladan bir kolon, fazladan bir index ve hiçbir kazanç demektir. ★ **Kural körlemesine uygulanmaz** — "her tabloda UUID" bir alışkanlık, gerekece değil.

★ **Neden tablo, neden kod içinde sabit liste değil:** Tatil listesi **veridir, kod değildir**. Kod içine gömülürse her yıl kod değiştirip yeniden yayına çıkmak gerekir — ve o yayını yapacak kişi tatil listesini bilmek zorunda kalır. Tabloda olunca yönetici yıl başında satır ekler.

⚠ **Yıllık bakım borcu:** Dini bayram tarihleri değiştiği için liste her yıl güncellenmeli. Bu, `docs/project/altyapi-durumu.md` dosyasına **bilinen bakım işi** olarak yazılır — sessiz bırakılırsa bir Ocak sabahı SLA'lar sessizce yanlış hesaplanır.

3.11 `work_order_attachments` — 🚫 **AÇILMIYOR, tasarımı hazır** ★ V11

Ödevde dosya/fotoğraf eki **yok** (yalnızca yorum isteniyor). 🚫 Kullanılmayan tablo açmak ödev §32'nin "gereksiz teknoloji, pattern, katman veya abstraction eklemek tek başına olumlu değerlendirilmez" maddesine takılır.

★ **Ama tasarımı burada hazır** — istenirse **tek migration + tek modül**:

Kolon	Tip	Not
<code>id</code>	uuid	
<code>work_order_id</code>	uuid	→ <code>work_orders</code> , CASCADE
<code>uploaded_by</code>	uuid	→ <code>users</code> , RESTRICT
<code>file_name</code>	varchar(255)	Kullanıcının gördüğü ad
<code>storage_key</code>	varchar(500)	🚫 Dosyanın kendisi veritabanında durmaz — nesne deposundaki anahtar durur
<code>mime_type</code>	varchar(100)	🚫 İstemcinin beyanına güvenilmez, sunucuda doğrulanır
<code>size_bytes</code>	int	Boyut sınırı kontrolü
<code>checksum_sha256</code>	varchar(64)	Mükerrer yükleme tespiti + bütünlük
<code>created_at</code>	timestampz	

⚠ **Eklenirse doğacak işler:** nesne deposu (S3/MinIO) · dosya türü ve boyut doğrulaması · virüs taraması kararı · 🚫 **KVKK:** fotoğrafta insan olabilir → kişisel veri envanterine yeni satır, saklama süresi ve erişim kuralı gerekir. ★ Bu yüzden "sadece tablo eklemek" değil, **yarım günlük bir iş**.

4. Index kararları ✅ (§11 "hangi alanlara index eklendiği, composite index tercihleri")

⚠ **Index bedava değildir:** her yazma işleminde güncellenir ve yer kaplar. Bu yüzden "her kolona index" yanlıştır; index **fiilen koşulan sorgulara** göre konur.

Tablo	Index	Hangi sorgu için
users	email benzersiz	Giriş
	employee_no benzersiz	Sicil çıkışması
	(role, is_active)	★ Atama ekranı: "aktif teknik personel"
locations	code benzersiz	Kod çıkışması
	(is_active, deleted_at)	Liste varsayılan filtresi
	name GIN + pg_trgm	Arama
assets	code benzersiz	
	(location_id, operational_status)	Varlık listesinin en sık filtre çifti
	criticality	Kritik varlık raporu
	(name, code) GIN + pg_trgm	Arama
work_orders	number benzersiz	Numaradan arama
	status	Liste ekranının en sık filtresi
	(assignee_id, status)	★ "Bana atananlar" — teknik personelin ana ekranı
	(location_id, status)	Lokasyon + durum birlikte filtreleniyor
	asset_id	Varlık detayındaki iş emri geçmişi
	requester_id	"Kendi taleplerim"
	created_at DESC	Varsayılan sıralama
	sla_due_at	"SLA'sı yaklaşanlar" filtresi
	★ V7 kısmi index: sla_due_at WHERE açık ve ihlal işaretlenmemiş	SLA ihlal taraması (5 dakikada bir)
	(title, description) GIN + pg_trgm	Metin araması ✅ §17
work_order_events	(work_order_id, created_at)	Detay ekranı zaman çizelgesi
work_order_comments	(work_order_id, created_at)	Yorum akışı
	(work_order_id, is_internal)	Talep sahibine dahili notsuz liste
notifications	dedupe_key benzersiz	★ 12 Mükerrer engeli
	(user_id, is_read, created_at DESC)	Zil ikonu + bildirim listesi

Tablo	Index	Hangi sorgu için
daily_operation_summaries	summary_date benzersiz	★ 17 İdempotency
refresh_tokens	token_hash benzersiz	Yenileme
	(user_id, revoked_at)	"Tüm oturumları kapat"

i Birleşik (composite) index'te SIRA önemlidir

(location_id, status) index'i şu sorgularda çalışır:

- WHERE location_id = ? ✓
- WHERE location_id = ? AND status = ? ✓

Şunda **çalışmaz**: WHERE status = ? 🚫 — çünkü index telefon rehberi gibidir: soyada göre sıralı bir rehberde "adı Ahmet olanları" bulamazsınız.

★ Bu yüzden status ayrıca tek başına index'lendi: liste ekranında en sık kullanılan tekil filtre o.

i ★ V7 kısmi index — neden gerekli ve Prisma'da neden elle yazılıyor

SLA ihlal taraması şu sorguyu 5 dakikada bir koşuyor: "süresi geçmiş + hâlâ açık + henüz ihlal işaretlenmemiş".

Normal bir sla_due_at index'i tüm iş emirlerini taşır — iki yıl sonra kapatılmış on binlerce kapalı kayıt da dahil. Kısmi index yalnızca aday satırları tutar: küçük kalır, önbellekte durur, tarama hızlı biter.

```
-- migration'a ELLE yazılıyor: Prisma şema dilinde "WHERE" karşılığı yok
CREATE INDEX work_orders_sla_scan_idx
ON work_orders (sla_due_at)
WHERE is_sla_breached = false
AND status NOT IN ('CLOSED', 'CANCELLED');
```

⚠ **Dürüst not:** Prisma bu index'i şemadan üretemediği için migration dosyasına elle eklenir ve prisma migrate diff karşılaştırmasında "şemada yok" uyarısı verebilir. Bu bilinçli bir sapmadır ve ADR'ye yazılır.

5. Silme davranışları ✅ (§11 "cascade davranışları")

İlişki	Davranış	Neden
<code>assets.location_id</code> → <code>locations</code>	RESTRICT	Varlığı olan lokasyon silinemez — sessiz veri kaybı olmasın
<code>work_orders.location_id</code> → <code>locations</code>	RESTRICT	Aynı
<code>work_orders.asset_id</code> → <code>assets</code>	RESTRICT	Aynı
<code>work_orders.requester_id</code> → <code>users</code>	RESTRICT	Talebi açan kişi kaydı yok olamaz
<code>work_orders.assignee_id</code> → <code>users</code>	RESTRICT	Aynı
<code>work_order_comments.author_id</code> → <code>users</code>	RESTRICT	Yorumun yazarı kaybolamaz
<code>work_order_events.work_order_id</code> → <code>work_orders</code>	CASCADE	Olay iş emri olmadan anlamsız
<code>work_order_comments.work_order_id</code> → <code>work_orders</code>	CASCADE	Aynı
<code>notifications.work_order_id</code> → <code>work_orders</code>	CASCADE	Aynı
<code>notifications.user_id</code> → <code>users</code>	CASCADE	Bildirim alıcısı olmadan anlamsız
<code>refresh_tokens.user_id</code> → <code>users</code>	CASCADE	Jeton sahibi olmadan anlamsız

★ **Genel ilke:** "Bu kayıt tek başına bir anlam taşıyor mu?" Taşıyorsa **RESTRICT** (silinmesin, önce ilişkisi çözülsün); taşımıyorsa **CASCADE**.

⚠️ **Pratikte CASCADE'ler neredeyse hiç tetiklenmez** — ana kayıtlar yumuşak siliniyor (§1.6), yani gerçek **DELETE** çalışmıyor. Yine de tanımlanıyor: (a) niyet beyanıdır, (b) test temizliği ve KVKK imha işleminde gerçek silme kullanılır, (c) tanımsız bırakılırsa PostgreSQL varsayılanı **NO ACTION** olur ve sürpriz hata verir.

6. Kişisel veri envanteri (KVKK)

⚠️ **Bu projede tüm veri sahtedir** ✅ (§5.1 gerçek kişisel veriyi yasaklıyor). Envanter yine de gerçekmiş gibi çıkarıldı: sonradan eklemek pahalı, baştan kurmak bedava.

Alan	Neden toplanıyor	Ne kadar tutulur	Kim erişir	Şifreli mi
Ad, soyad	İş emrinde "kim açtı / kime atandı" görünmeli	Personel kaydı aktif olduğu sürece + §7	Tüm giriş yapmış kullanıcılar (yalnızca ad-soyad)	🚫
Kurum e-postası	Giriş kimliği	Aynı	Yalnızca yönetici; diğer rollere dönülmez	🚫
Şifre	Kimlik doğrulama	Aynı	🚫 Hiç kimse — geri çevrilemez argon2id özeti	✅ tek yönlü
İş telefonu	Saha personeline ulaşmak	Aynı	Operasyon sorumlusu + yönetici	🚫
Birim	İş yükü dağılımını okumak	Aynı	Operasyon + yönetici	🚫
Sicil numarası	Kurum içi eşleştirme	Aynı	Yalnızca yönetici	🚫
Yorum metni	İş emri iletişimi	İş emriyle birlikte, süresiz	İlgili taraflar; dahili notu talep sahibi göremez ★ 11	🚫
Oturum jetonu özeti	Oturum yenileme	Azami 7 gün	🚫 Hiç kimse	✅ SHA-256

🚫 **Loglara asla yazılmayacaklar** ✅ (§25): şifre · erişim jetonu · yenileme jetonu · çerez içeriği · istek gövdesinin tamamı · e-posta adresi (★ *ek tedbir: log satırında kullanıcı **kimliği** yeterli, e-posta gereksiz*).

★ **Toplanmayanlar ve gerekçeleri** PRD §5.11'de. Özet: ev adresi, T.C. kimlik no, doğum tarihi, cinsiyet, fotoğraf, konum, IP adresi.

7. Saklama süreleri ve imha

⚠️ **Bu tablo bir hukuki görüş değildir** — kurumun KVKK birimi tarafından onaylanmalı (? 6). Buradaki süreler yaygın uygulamaya göre önerilmiştir.

Tablo	Süre	Süre dolunca	Süreyi zorunlu kılan
<code>work_orders</code> + <code>work_order_events</code> + <code>work_order_comments</code>	⚠ 10 yıl	★ Silinmez — kişisel alanlar zaten kullanıcıya bağlı , kullanıcı kaydı kapatılınca ad görünmez olur	⚠ Kurumsal faaliyet kaydı; genel zamanaşımı süresi (TBK m.146) yaygın referans
<code>users</code>	Personel ayrıldıktan sonra ⚠ 10 yıl	Kişisel alanlar silinir/maskelenir, satır kalır	⚠ İş Kanunu m.75 özlük dosyası yükümlülüğü ile ilişkili
<code>notifications</code>	⚠ 1 yıl	Silinir	Yok — operasyonel veri
<code>refresh_tokens</code>	Süresi dolunca + 30 gün	Silinir	Yok
<code>daily_operation_summaries</code>	⚠ 5 yıl	Silinir	Yok — rapor verisi
<code>holidays</code>	Süresiz	—	Kişisel veri değil

- **Personel kaydı kapatılınca:** Ad, soyad, e-posta, telefon, sicil no, birim → silinir/maskelenir. Oturumların tamamı kapatılır. 🚫 İş emirleri, olaylar ve yorumlar **silinmez** — bunlar kurumsal faaliyet kaydıdır; kişi bağlantısı kalıcı kimlik numarası üzerinden korunur, ad hiçbir ekranda görünmez.
- **İmha görevi:** ⚠ Periyodik imha **altı ayı geçemez** (Kişisel Verilerin Silinmesi, Yok Edilmesi veya Anonim Hale Getirilmesi Hakkında Yönetmelik m.11). 🚫 Bu proje kapsamında **otomatik imha işi yazılmıyor** (ödevde yok); gerekliliği burada kayıtlı — bilinen teknik borç olarak `README` 'ye girer.

🚫 **"Kişisel alanları boşaltıp satırı bırakmak" anonimleştirme DEĞİLDİR.** Satır hâlâ bir kullanıcı kimliği üzerinden başka kayıtlara bağlı olduğu için yapılan şey **takma adlaştırmadır** (pseudonymization). Doğru ifade: *"kişisel alanlar silinir, faaliyet kayıtları kişiselleştirilmeden saklanır."*

8. Ödev §11'in istediği kararlar — kontrol listesi

★ Ödev §11, dokümantasyonda **en az** şu kararların açıklanmasını istiyor. Teslimden önce bu tablo tek tek doğrulanır.

#	Ödevin istediği karar	Cevap	Nerede
1	Tabloların ve ilişkilerin neden bu şekilde tasarlandığı	Her tablo bir PRD iş kuralından doğdu; §2.2'de eşleme	§2, §3
2	Hangi alanlara index eklendiği	25 index, her biri bir sorguya bağlı	§4
3	Composite index tercihleri	<code>(assignee_id,status)</code> , <code>(location_id,status)</code> , <code>(user_id,is_read,created_at)</code> — sıra gerekçesiyle	§4
4	Enum'ların nasıl saklandığı	PostgreSQL yerel enum, değer metin ★ V2	§1.5
5	Soft delete kullanılan yapılar	<code>users</code> , <code>locations</code> , <code>assets</code> — ve neden diğerlerinde yok ★ V3	§1.6
6	Silme davranışları	RESTRICT/CASCADE ayrımı tek ilkeye bağlı	§5
7	Audit yaklaşımı	Dört alan + merkezî eklenti; audit ≠ geçmiş ayrımı	§1.4
8	Concurrency yönetimi	<code>version</code> kolonu + koşullu güncelleme → 409	§1.7
9	Tarih ve saat standardı	<code>timestampz</code> + UTC; salt tarih alanları <code>date</code>	§1.3
10	İş emri numarası üretme yöntemi	PostgreSQL sequence + <code>IE-YYYY-NNNNNN</code> ★ 13	PRD §5.4
11	Bildirim tekrarlarını önleme yöntemi	<code>dedupe_key</code> benzersiz index ★ 12	§3.8
12	Nullable alanlar	<code>null</code> tek anlam taşır; boolean asla null olmaz	§1.8
13	Primary key / foreign key / unique constraint	UUID v7 ★ V1; FK'ler §5'te; benzersizler her tabloda	§1.2, §5
14	İşlem geçmişi tabloları	<code>work_order_events</code> + <code>work_order_comments</code> , ikisi de değişmez	§3.6, §3.7

★ **Ayrıca zorunlu:** `docs/database.dbml` — 🚫 elle yazılmaz, migration'ın ürettiği **gerçek şemadan** `@dbml/cli` ile üretilir ve CI fark görürse build kırmızı yanar (§10).

9. Sahte veri planı

🚫 **Gerçek kişisel veri kullanılmaz** ✅ (§5.1). 🚫 Gerçek ad, gerçek telefon, gerçek e-posta, gerçek kimlik numarası tohumlanmaz — depo herkese açık olabilir.

9.1 Neden bu kadar veri gerekiyor

Boş ekran hiçbir şeyi doğrulamaz. ★ Ancak gerçekçi veriyle görünen şeyler: sayfalamanın ikinci sayfası · uzun metnin taşması · "boş liste" ile "veri var" farkı · sıralamanın kararlılığı · filtre kombinasyonlarının boş

dönmesi · pano sayılarının anlamlı çıkması · SLA renk kodlarının hepsinin görünmesi.

9.2 Kurallar

Kural	Nasıl sağlanıyor
İdempotent	Sabit kimliklerle <code>upsert</code> — iki kez çalıştırınca veri ikilenmez §9.3
Deterministik	Rastgelelik sabit tohumla üretilir (<code>faker.seed(20260826)</code>); aynı komut aynı veriyi üretir. 🚫 Yoksa testler kararsızlaşır
Ayırt edilebilir	★ 22 <code>is_seed_data = true</code> · e-postalar <code>@ornek.test</code> alan adında (★ <code>.test</code> RFC 2606 ile ayrılmış, gerçekte kimseye ait olamaz)
Korumalı	★ V16 Tohumlama, <code>is_seed_data = false</code> olan tek bir satır görürse durur ve uyarır. Ayrıca <code>SEED_ALLOWED=true</code> çevre değişkeni şart
Şifre depoda yok	🚫 Demo şifre <code>.env</code> içinden okunur; yoksa tohumlama hata verir . Sessiz varsayılan şifre yok
Sınır durumları bilerek	§9.5'teki liste — mutlu yol tek başına arayüzü doğrulamaz

9.3 Kimlikler nasıl sabitleniyor

★ İdempotent `upsert` için kimliğin **önceden bilinmesi** gerekir. Rastgele UUID üretilirse ikinci çalıştırma aynı kaydı bulamaz ve kopyasını yaratır.

```
seedId("user", 3)    → sabit bir UUID (aynı girdi → hep aynı çıktı)  
seedId("asset", 42)  → sabit bir UUID
```

★ Yöntem: sabit bir ad alanı (namespace) + tür + sıra numarası → **UUID v5** (içerikten türetilen kimlik).
Böylece hem deterministik hem çakışmasız.

9.4 Zaman nasıl kurgulanıyor ★ V15

🚫 Sabit tarihlerle tohumlanan demo bir ay sonra **bayatlar**: "SLA'sı bugün dolacak" listesi boşalır, pano sıfır gösterir.

★ Çözüm: tüm tarihler `SEED_NOW` 'a **görelî** üretilir.

Kayıt	Zaman kurgusu
Kapatılmış iş emirleri	<code>SEED_NOW - 180...10 gün</code> arasına yayılır
Açık/işlemdeki iş emirleri	<code>SEED_NOW - 20 gün ... SEED_NOW - 1 saat</code>
SLA'sı yaklaşanlar	<code>sla_due_at = SEED_NOW + 30 dk ... + 6 saat</code>
SLA'sı geçmiş, işaretlenmemiş	<code>sla_due_at = SEED_NOW - 2 saat</code> ★ tarama işi ilk çalışmada bunları yakalamalı
Arşiv adayları	<code>closed_at = SEED_NOW - 190 gün</code> ★ (eşik 180)

⚠ `SEED_NOW` verilmezse `now()` kullanılır — **testlerde sabit değer verilir**, demo çalıştırmalarında verilmez. Böylece demo her gün taze, test her koşuda aynı.

9.5 Miktarlar

Tablo	Adet	Neden bu kadar
<code>holidays</code>	~15	2026 sabit tatilleri + ⚠ dini bayramlar (? 7 gelince)
<code>users</code>	23	1 yönetici · 2 operasyon · 8 teknik personel · 12 talep sahibi. 8 teknisyen → "personel bazında iş yükü" grafiği anlamlı; 23 satır sayfalamada (20) ikinci sayfayı doğurur
↳ <i>pasif kullanıcı</i>	2	★ Biri teknik personel: "pasif kullanıcıya atama yapılamaz" kuralı test edilebilsin. Diğeri talep sahibi: giriş engeli
<code>locations</code>	24	20 aktif + 4 pasif → pasif lokasyon kuralı görünür; 24 satır ikinci sayfayı doğurur
<code>assets</code>	120	24 lokasyona dağılır (ortalama 5). Her kritiklik ve her operasyonel durum yeterli sayıda temsil edilsin
<code>work_orders</code>	400	Sekiz durum × dört öncelik × dört tür kombinasyonlarında boş liste kalmamasın; sayfalama 20 sayfa üretir; pano sayıları anlamlı çıkar
<code>work_order_events</code>	~1600	Her iş emrinin gerçek durum yolculuğu üretilir — ortalama 4 olay
<code>work_order_comments</code>	~600	İş emri başına 0–5; ★ %20'si dahili not
<code>notifications</code>	~250	★ %40'ı okunmamış — okundu/okunmadı ayrımı ve zil sayacı görünsün
<code>daily_operation_summaries</code>	30	Son 30 gün — panoda eğilim görünsün

★ **Toplam ~3000 satır.** Tohumlama birkaç saniyede biter; bu bilinçli: geliştirici günde onlarca kez veritabanını sıfırlayacak.

9.6 Dağılımlar

🚫 Hepsi "aktif" olursa diğer durumların ekranı **hiç** test edilmez.

Alan	Dağılım	Neden
<code>work_orders.status</code>	<code>TALEP</code> %8 · <code>AÇIK</code> %12 · <code>ATANDI</code> %15 · <code>İŞLEMDE</code> %18 · <code>PARÇA BEKLİYOR</code> %7 · <code>ÇÖZÜLDÜ</code> %10 · <code>KAPATILDI</code> %25 · <code>İPTAL</code> %5	Gerçek bir kurumda kapalı işler çoğunluktadır; açık iş sayısı panoda anlamlı bir sayı vermeli
<code>work_orders.priority</code>	Düşük %20 · Normal %45 · Yüksek %25 · Kritik %10	Kritik az olmalı — ★ hepsini kritik yapmak SLA renklerini anlamsızlaştırır
<code>work_orders.type</code>	Arıza %55 · Planlı bakım %25 · Periyodik kontrol %15 · Kurulum %5	★ Üç SLA politikasının üçü de yeterli sayıda örnekle test edilsin
<code>is_sla_breached</code>	Açık işlerin ~%15'i	Panodaki "SLA ihlali" sayısı ne sıfır ne de çoğunluk olmalı
<code>assets.criticality</code>	Düşük %20 · Normal %45 · Yüksek %25 · Kritik %10	Çarpanların hepsi kullanılсын ($\times 1.5$ / $\times 1$ / $\times 0.75$ / $\times 0.5$)
<code>assets.operational_status</code>	Çalışıyor %70 · Arızalı %10 · Bakımda %8 · Kullanım dışı %7 · Hurda %5	★ Son ikisi iş emri açmayı engelliyor — kural test edilebilsin
<code>work_order_comments.is_internal</code>	%20	Dahili not görünürlük kuralı test edilsin
<code>notifications.is_read</code>	%60 okundu · %40 okunmadı	Zil sayacı sıfır olmasın
<code>locations.is_active</code>	%83 aktif · %17 pasif	Pasif lokasyon kuralı

9.7 Bilerek eklenen sınır durumları ★

🚫 Mutlu yol tek başına hiçbir şey kanıtlamaz. Aşağıdakiler **kasıtlı**:

#	Ne	Neyi sınıyor
S1	200 karakterlik başlık + 4000 karakterlik açıklama	Tablo hücrelerinde taşma, detay ekranında kaydırma
S2	Tek karakterlik ad, Türkçe karakterli uzun lokasyon adı (Şişli Çağlayan Müdürlüğü Ek Hizmet Binası)	Sıralama, arama, hizalama
S3	Atanmamış açık iş emri	"Atanmamış iş işleme alınamaz" kuralı
S4	SLA'sı 30 dakika sonra dolacak kritik iş	Hatırlatma işi ve uyarı rengi
S5	SLA'sı geçmiş ama işaretlenmemiş iş	★ Tarama işi ilk çalışmada bunu yakalamalı — işin gerçekten çalıştığının kanıtı
S6	Parça bekleyen iş emri	★ 8 duraklatma alanları dolu olsun
S7	0 yorumlu ve 12 yorumlu iş emri	Boş durum + uzun akış
S8	Pasif lokasyonda geçmiş iş emri	"Pasifleştirme geleceği kapatır, geçmişi değil" kuralı
S9	Hurdaya ayrılmış varlık + o varlığın eski iş emirleri	Yeni iş emri engeli, geçmiş korunuyor
S10	Pasif teknik personel + üzerinde açık iş emri	Atama kuralı ve "pasif kullanıcının açık işi ne olacak" sorusu
S11	190 gün önce kapatılmış iş emirleri	Arşiv adayı işi
S12	Aynı saniyede oluşturulmuş iki iş emri	★ Sıralama kararlılığı: eşit created_at 'te ikincil sıralama alanı olmazsa sayfalar arasında kayıt kaybolur veya tekrar eder
S13	Yalnızca dahili notu olan iş emri	Talep sahibi ekranında "yorum yok" görmeli
S14	Aynı varlığa açık iki iş emri	★ Ödev bunu yasaklamıyor — ama ekranda karışıklık yaratmadığı görülmeli

9.8 Demo hesaplar

Rol	E-posta	Şifre
Yönetici	yonetici@ornek.test	🚫 .env → SEED_DEFAULT_PASSWORD
Operasyon Sorumlusu	operasyon@ornek.test	Aynı
Teknik Personel	teknisyen@ornek.test	Aynı
Talep Sahibi	talep@ornek.test	Aynı

- 🚫 **Gerçek şifre depoya yazılmaz.** Değer **.env** dosyasından okunur, **.env.example** içinde yalnızca **anahtar adı** durur ✅ (§5.1, §26).
- ★ Şifresi olan hesap **dört tane** — her rolden bir tane. Kalan 19 kullanıcının şifresi **bilerek** aynı değerle üretiliyor ama giriş yapmaları beklenmiyor; onlar listelerde ve atamalarda **isim** olarak var.

-  Production'da demo hesap **açılmaz** — tohumlama zaten §9.2'deki kapıdan geçemez.

9.9 Tohumlama sırası

-  Sıra keyfi değil: her tablo, yabancı anahtarla bağlı olduğu tablodan **sonra** gelir.

1. holidays (bağımsız)
2. users ★ önce yönetici – created_by zinciri ondan başlar
3. locations
4. assets (locations'a bağlı)
5. work_orders (locations + assets + users'a bağlı)
6. work_order_events ★ her iş emrinin durum yolculuğu üretilir
7. work_order_comments
8. notifications (users + work_orders'a bağlı)
9. daily_operation_summaries ★ son 30 gün, work_orders'tan HESAPLANARAK

★ **9. adım özellikle önemli:** günlük özetler uydurulmuyor, tohumlanan gerçek iş emirlerinden **hesaplanıyor**. Böylece pano ile liste ekranı aynı sayıyı gösterir; uydurulsaydı ikisi tutmaz ve "*hangisi doğru*" sorusu doğardı.

9.10 Tohumlama sonrası doğrulama

- ★ Tohumlama "çalıştı" demek yetmez — şunlar **kontrol edilir**:

Kontrol	Beklenen
Toplam satır sayıları	§9.5'teki miktarlarla birebir
<code>created_by</code> boş satır	★ V14 yalnızca ilk yönetici ; başka satır varsa hata
Yetim kayıt	Yok — her <code>work_order_events</code> satırı bir iş emrine bağlı
<code>is_seed_data = false</code> satır	 Sıfır olmalı
Aynı <code>dedupe_key</code>	 Sıfır çakışma (zaten benzersiz index engelliyor)
Durum–zaman tutarlılığı	★ <code>closed_at</code> dolu olan her satırın durumu KAPATILDI ; <code>resolution</code> dolu olmayan ÇÖZÜLDÜ satırı yok
Tarih tutarlılığı	<code>sla_start_at ≤ sla_due_at</code> · <code>created_at ≤ resolved_at ≤ closed_at</code>
Pano ile liste	Aynı sayıyı gösteriyor

⚠ Bu kontroller **entegrasyon testine** dönüştürülür: tohumlama bozulursa CI kırmızı yanar.  Elle bakılan kontrol, üçüncü haftada bakılmayan kontroldür.

10. DBML ve migration ilişkisi ✅ (§11 — docs/database.dbml zorunlu)

🚫 **DBML dosyası elle yazılmaz.** Elle yazılan doküman ilk şema değişikliğinde eskir ve kimse fark etmez.

```
schema.prisma —migrate→ PostgreSQL (gerçek şema)
                        |
                        | dışa aktar (yalnızca yapı, veri değil)
                        ▼
                  @dbml/cli → docs/database.dbml
                        |
                        CI: depodaki dosyayla KARŞILAŞTIR
                        fark varsa → build KIRMIZI
```

★ Böylece ödevin "*DBML dosyası ile migration'lar birbiriyle uyumlu olmalıdır*" şartı **iddia değil, kapı** hâline geliyor: uyumsuzluk fark edilmeyi beklemiyor, birleştirmeyi **durduruyor**.

11. Bu belge nereye bağlanıyor

Sonraki iş	Neyi buradan alıyor
Adım 3 — Veri modeli	§1–§5 doğrudan <code>schema.prisma</code> ya çevrilir
Adım 3 — Tohumlama	§9'un tamamı
Adım 6 — İş emri çekirdeği	§3.5, §3.6 kolonları ve kısıtları
Adım 7 — SLA	§3.5 C bölümü (SLA kolonları) + §3.10 tatil tablosu
Adım 8 — Arka plan işleri	★ V7 kısmi index + §3.8 <code>dedupe_key</code> + §3.9 özet tablosu
Teslim — docs/database-decisions.md	§8 kontrol listesi + her ★ kararın gerekçesi
Teslim — docs/database.dbml	§10
Sunum	§3.6'daki "neden tek olay tablosu" ve §4'teki index gerekçeleri — canlı incelemede en çok sorulacak iki yer